



UNIFEI

UM FRAMEWORK ORIENTADO À SERVIDORES VOLUNTÁRIOS PARA WEB
HOSTING DISTRIBUÍDO

**Revisão bibliográfica e plano de trabalho enviado
para análise e deliberação da banca de TCC01**

Aluno(s):

Daniel Ferreira Lara – ECO

Dielson Soares de Oliveira Junior - ECO

Orientação:

Dr. Enzo Seraphim - Orientador

Dra. Thatyana de Faria Piola Seraphim - Orientadora

5 de dezembro de 2025

Sumário

1	Introdução	3
2	Motivação	4
2.1	Monopolização digital	4
2.2	Dependência de modelos de negócio	4
2.3	Aplicabilidade	4
2.4	Tendências do mercado	5
2.5	Motivações dos autores	5
3	Revisão bibliográfica	6
3.1	Computação voluntária	6
3.2	Balanceadores de carga	6
3.2.1	Contexto de sistemas distribuídos	7
3.2.2	Métodos estáticos e dinâmicos	7
3.2.3	Distribuição de balanceamento	8
3.2.4	Soluções de Mercado	8
3.3	Blockchain	9
3.4	Trabalhos acadêmicos similares	10
3.5	Soluções de Mercado e DePINs	11
3.5.1	Computação e Hospedagem Descentralizadas	11
3.5.2	Armazenamento e Distribuição	12
4	Objetivos específicos	14
4.1	Entregas	14
4.1.1	Serviço web para testes	14
4.1.2	Balanceador de carga configurado	14
4.1.3	Sistema de inscrição de voluntários	14
4.1.4	Túnel principal-voluntário e sua arquitetura	15
4.1.5	Extra: Sistema de validação e sua arquitetura	15
4.1.6	Extra: Banco de dados com proteção contra leitura e sua arquitetura	15
4.2	Experimentos e Pesquisas	15
4.2.1	Pesquisa de interesse e confiabilidade	15
4.2.2	Comparação de diferentes algoritmos de balanceamento	15
4.2.3	Comparação de teste de carga com soluções alternativas	16

5	Materiais e métodos	17
5.1	Materiais e ferramentas	17
5.1.1	<i>Hardware</i> e ambiente de virtualização	17
5.1.2	Software e tecnologias	17
5.2	Procedimentos metodológicos	17
5.2.1	Pesquisa de interesse e confiabilidade	17
5.2.2	Experimentos de balanceamento e comparação	18
6	Resultados esperados	19
6.1	Pesquisa de interesse e confiabilidade	19
6.2	Experimentos de balanceamento de carga	19
6.3	Comparação entre arquiteturas	19
6.4	Métricas gerais previstas	19
7	Cronograma de atividades do TCC02	20

1 Introdução

A arquitetura original da Internet foi concebida sob princípios de descentralização e comunicação sem pontos únicos de controle, a evolução da web nas últimas décadas culminou em uma centralização massiva da camada de infraestrutura. Atualmente, um oligopólio formado por provedores de nuvem em hiperescala como AWS, GCP e Microsoft Azure detém o controle majoritário dos recursos mundiais de computação e armazenamento em nuvem. Embora esse modelo ofereça conveniência e baixa latência, ele cria dependência tecnológica e estabelece barreiras financeiras para entidades sem fins lucrativos, instituições de ensino e projetos sociais.

Em contrapartida, existe uma vasta capacidade computacional ociosa distribuída nas bordas da rede, em computadores pessoais e servidores institucionais, que permanece subutilizada. O paradigma da Computação Voluntária, popularizado por projetos científicos como o BOINC, demonstrou a viabilidade de agregar esses recursos dispersos. Posteriormente, o surgimento de *blockchains* e Redes de Infraestrutura Física Descentralizada (DePINs), provou-se possível utilizar essa infraestrutura descentralizada para hospedagem de serviços *web*. No entanto, a maioria das soluções de DePIN modernas baseiam-se em modelos econômicos complexos de criptoativos (*tokenomics*) introduzindo fricção financeira que dificulta a adoção por atores não comerciais.

Este trabalho propõe o desenvolvimento de um **Framework Orientado a Servidores Voluntários para Web Hosting Distribuído**. O objetivo central é preencher a lacuna entre a tecnologia das DePINs e o modelo colaborativo acessível da Computação Voluntária.

Para validar a proposta, será desenvolvida uma implementação de referência utilizando o balanceador de carga HA-Proxy, um sistema de túneis seguros e outros componentes intermediários avaliando-se posteriormente o desempenho e a latência em comparação com arquiteturas tradicionais de nuvem. O resultado esperado é uma nova solução de infraestrutura de hospedagem, contribuindo com serviços *web* não contemplados pelas arquiteturas existentes.

O restante deste trabalho está organizado da seguinte forma: a Seção 2 detalha a motivação e o contexto de mercado; a Seção 3 revisa a literatura sobre computação voluntária, balanceamento de carga e arquiteturas descentralizadas similares; a Seção 4 define os objetivos específicos e a nossa arquitetura proposta; a Seção 5 descreve a metodologia experimental e os materiais que serão utilizados; a Seção 6 apresenta os resultados esperados e métricas de validação e a Seção 7 apresenta o cronograma completo da implementação do projeto.

2 Motivação

2.1 Monopolização digital

O cenário digital contemporâneo é marcado por um paradoxo estrutural: enquanto a internet foi concebida como descentralizada, a camada de aplicação se solidificou num sistema centralizador. A infraestrutura global de computação e armazenamento gira em torno de provedores de nuvem em hiperescala, notavelmente AWS (Amazon Web Services), GCP (Google Cloud Platform) e Microsoft Azure, que detêm o controle majoritário dos recursos mundiais, somando 65% da parcela de mercado em 2018 e 64% em 2025 [1] [2]. Esta centralização, apesar de proporcionar conveniência e baixa latência, impõe custos sistêmicos e dependência ao provedor, criando barreiras financeiras para, por exemplo, entidades sem fins lucrativos.

2.2 Dependência de modelos de negócio

O modelo de financiamento coletivo é o mais comum para sustentar projetos sem apelo financeiro explícito. Exemplos são a Wikipedia, o Internet Archive e o OpenStreetMap, que regularmente organizam campanhas de arrecadação financeira coletiva [3] [4] [5]. No entanto, os modelos de financiamento público têm limitações: por exemplo, o Internet Archive terminou o ano fiscal de 2023 em uma dívida de quase 9 milhões de dólares [4]. Modelos híbridos podem prover uma estabilidade fiscal, sendo um exemplo a Mozilla Foundation, uma organização sem fins lucrativos, dona do navegador Mozilla Firefox, um projeto com fins lucrativos, responsável pela maior parte da receita da fundação mediante *royalties* de motores de pesquisa [6].

Muitas razões podem ser atribuídas para uma pequena parcela de indivíduos doarem para iniciativas como as citadas anteriormente. Um relatório sobre filantropia nos EUA descobriu que o impedimento mais comum para indivíduos doarem é a alegação de que o valor não se encaixa em seu orçamento. Essa perspectiva se faz coerente, uma vez que as porções populacionais de maior renda apresentam maior interesse em doar. [7]

Além de empresas e indivíduos, outra força de impacto e interesse são instituições de ensino. Universidades apoiam o desenvolvimento e o financiamento de projetos de interesse social e acadêmico. No entanto, essas instituições são limitadas por uma austeridade fiscal maior, e menor grau de liberdade financeira que instituições privadas.

Então, no estado atual, hospedagem de serviços *web* complexos são impraticáveis sem um modelo de negócio. Para tal, os serviços são limitados a interesses econômicos, excluindo da internet projetos de valor social sem apelo financeiro. Os poucos serviços sobreviventes desse mercado são obrigados a equilibrar um balanço fiscal apertado [3] [4] [5].

Neste cenário, vê-se a necessidade de eliminar ou reduzir a limitação financeira. Instituições de ensino geralmente tem o próprio poder computacional que provavelmente se encontra parcialmente ocioso, exceto por períodos específicos e curtos de altas no tráfego, como o processamento de matrículas. De forma equivalente, indivíduos também são capazes de dispor uma pequena parcela de computação igualmente ociosa em seus computadores pessoais. Ao passar o apoio a serviços públicos de uma esfera financeira para uma esfera de poder computacional inutilizado, eliminamos algumas barreiras ao apoio.

2.3 Aplicabilidade

É importante notar que apesar de **distribuído**, nosso framework não tem intenção de ser **descentralizado**. Essa diferença é tão limitadora quanto inovadora em termos de possíveis aplicações, como sistemas distribuídos voluntários web descentralizados são relativamente maduros.

Em uma direção de propósito diferente, a arquitetura discutida nesse trabalho tem a intenção de simplificar o processo de construção de uma infraestrutura confiável utilizando uma base de voluntários.

Um exemplo de uma utilização da arquitetura proposta é uma plataforma de jornais universitários. Em uma configuração normal, seria necessário que uma universidade se disponha a hospedar o sistema completo ou que o sistema seja hospedado na nuvem, tornando-se refém de doações de empresas, da arquitetura e precificação arbitrária do provedor, ou da criação de um modelo negócio. Dentro da arquitetura orientada à voluntários, a partir de um servidor central de baixo custo, universidades, indivíduos e empresas podem contribuir para a hospedagem do sistema sem compromissos e portanto menor capacidade de imposição de responsabilidades.

2.4 Tendências do mercado

Em complemento aos motivos apresentados, projeções indicaram no início de 2025 um aumento no gasto com infraestrutura no mundo de 19% em relação aos 321.3 bilhões de US\$ de 2024 [2]. Um relatório da S&P Global também indicou uma tendência na pesquisa feita, observaram um aumento dos custos de infraestrutura em 60% das empresas contactadas, destacando uma estratégia de mitigação de custos voltada à repartição dos serviços entre as líderes de mercado [8].

2.5 Motivações dos autores

Os dois autores tiveram experiências de mercado na área de infraestrutura em escala e observaram uma lacuna em métodos acessíveis, flexíveis e escaláveis de construção de uma infraestrutura. O projeto, além da relevância acadêmica e social, é peça central de projetos subsequentes dos autores, que dependem de uma arquitetura como a descrita.

3 Revisão bibliográfica

3.1 Computação voluntária

O conceito de computação voluntária não é recente e tampouco se conecta originalmente com infraestrutura de serviços web. O primeiro projeto de computação voluntária em escala foi o *Great Internet Mersenne Prime Search* (GIMPS), uma busca colaborativa pelos Primos de Mersenne. Em novembro de 2025, o projeto GIMPS conta com mais de 3 milhões de computadores registrados e cerca de 23,5 mil computadores ativos no último mês. [9]

Além do conceito inicial, dentre os vários avanços da computação voluntária, destaca-se o interesse na construção dos *middlewares* que são utilizados para conectar os voluntários aos projetos centrais. O *middleware* mais difundido é o *Berkeley Open Infrastructure for Network Computing* (BOINC). O BOINC conecta servidores de projetos de pesquisa com servidores (mais geralmente, computadores pessoais) voluntários que efetuarão uma porção dos cálculos necessários para a pesquisa. Os voluntários requisitam ao servidor de projeto uma carga de trabalho, efetuam os cálculos e enviam os resultados de volta. [10]

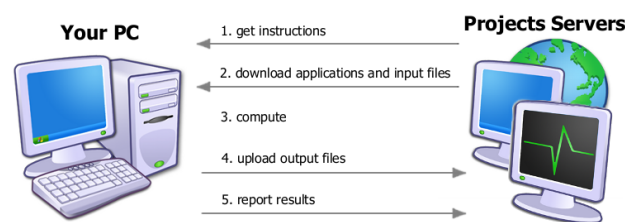


Figura 1: Função do middleware BOINC [10]

Para a nossa arquitetura, temos interesse em um fluxo de informações similar, mas na direção contrária, com as requisições *web* partindo do servidor central para o servidor voluntário.

Outro ponto de interesse dos estudos de computação voluntária são os sistemas de confiabilidade de resultados. A primeira questão que se pode ter sobre um sistema baseado em voluntários é se os dados podem sofrer manipulações. Cada *middleware* tem sua proposta para resolver esse problema, porém, de maneira geral, todos se encontram orbitando a mesma solução: sistemas de reputação e redundância. Voluntários ganham reputação por enviarem resultados corretos e resultados são ditos corretos por consenso, ao se calcular o mesmo resultado em servidores voluntários distintos [11].

Contrariamente, definir a veracidade por consenso não é trivialmente aplicável à distribuição de sistemas web, afinal a latência adicionada pela redundância prejudica gravemente a performance. Diferente de resultados de cálculos no caso de projetos de pesquisa, os resultados de páginas web e banco de dados são conhecidos. Portanto, comparar os resultados dos voluntários com resultados reais se torna mais simples do que no caso de aplicações clássicas de computação voluntária. Deve-se adicionar que o conceito de reputação mantém-se interessante para apontar possíveis agressores ou prováveis erros acidentais.

3.2 Balanceadores de carga

Se em propósito o problema assemelha-se ao problema da computação voluntária, nosso problema se assemelha em configuração de atores ao problema que balanceadores de carga tenta resolver. Afinal, balanceadores de carga, no contexto de serviços web, distribuem requisições à servidores inseridos na sua rede. Alguns problemas técnicos que enfrentaremos são bem estudados e considerados maduros no contexto de balanceadores de carga.

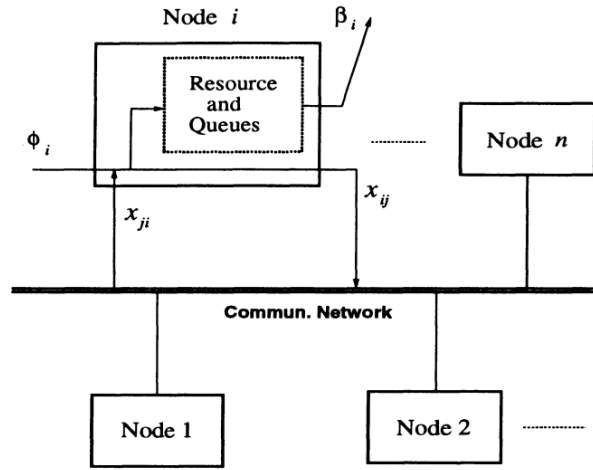


Figura 2: Diagrama de um sistema distribuído simples [12]

3.2.1 Contexto de sistemas distribuídos

É importante notar que os balanceadores de carga encaixam-se no contexto de sistemas distribuídos. Como visto na Figura 2, temos em um sistema distribuído pontos de computação (*nodes*, no nosso caso interpretados como servidores) i com taxas de processamento β_i , capazes de receber e enviar mensagens para outro ponto j qualquer com uma taxa ϕ_i . Para um ponto i , sua taxa β_i depende dos recursos disponíveis e a fila de processamento atual. [12]

O trabalho de um balanceador de carga em sistemas distribuídos é, então, adicionar tarefas às filas de processamento. Diferentes métodos se utilizam de parâmetros do sistema e dos pontos de computação, como β_i e ϕ_i , para otimizar as variáveis relevantes dessa comunicação. [12]

3.2.2 Métodos estáticos e dinâmicos

Para distribuir carga, deve-se ter conhecimento dos parâmetros de pontos de computação disponíveis. Métodos estáticos não buscam avaliar os estados instantâneos dos pontos e então assumem alguns dados sobre os estados a partir de dados anteriores, em uma média temporal ou fora de *runtime*, com dados estatísticos [12]. São mais facilmente aplicados em sistemas onde os pontos de comunicação são homogêneos e confiáveis e tarefas têm um tempo de processamento comparável. [13]

Algumas implementações [13] de balanceamento estático são:

1. *Round Robin*: carga é distribuída sequencialmente entre os pontos do primeiro ao último, retornando ao primeiro no fim de execução;
2. Escolha aleatória: pontos são escolhidos aleatoriamente;
3. *Central manager*: escolhe pontos com menor carga.

Adiante, métodos dinâmicos utilizam informações instantâneas para decidir o balanceamento de carga. Isto, logicamente, acarreta um *overhead* de processamento maior. Métodos dinâmicos constantemente tentam ajustar o balanceamento de acordo com as mudanças de carga dos pontos. [12]

Implementações comuns [13] para o balanceamento dinâmico são:

1. *Central queue*: uma fila central é mantida e pontos solicitam uma tarefa quando desocupados;
2. *Local queue*: filas locais são mantidas e pontos solicitam para pontos aleatórios uma tarefa quando desocupados.

3.2.3 Distribuição de balanceamento

O balanceador de cargas pode ser: centralizado (um único ponto é responsável pelo balanceamento), distribuído (múltiplos pontos são responsáveis pelo balanceamento) ou hierárquico (cria-se uma hierarquia de pontos de acordo com um critério, exemplo: posição geográfica). [13]

No contexto deste trabalho, distribuir o balanceador de cargas para os voluntários não é trivial, pois adiciona a complexidade de expor os voluntários uns aos outros. No entanto, centralizar o balanceador de cargas nos expõe ao problema de baixa tolerância à falhas e algumas perdas de performance, além do peso computacional completo no servidor principal. Uma solução possível é interpretar que *alguns* voluntários seriam tão confiáveis quanto o próprio servidor principal e então efetivamente dividir o trabalho do servidor principal. Esta solução de repartição do servidor principal é tão poderosa para a performance e disponibilidade (em vários eixos, não limitada ao balanceador de carga) quanto pode ser perigosa para a segurança do sistema.

3.2.4 Soluções de Mercado

Para contextualizar a implementação teórica discutida anteriormente, analisamos três soluções de balanceamento de carga em alta no mercado: Nginx, Traefik e HAProxy.

3.2.4.1 Nginx

O Nginx é uma das soluções mais populares, senão a mais popular, para balanceamento de carga e proxy reverso, atuando frequentemente em arquiteturas centralizadas. Embora contenha a implementação de um algoritmo dinâmico, sua aplicação típica em balanceamento de carga baseia-se em métodos estáticos com o *Round Robin* como padrão. [14]

No contexto do nosso problema, o Nginx destaca-se pela simplicidade, mas possui limitações nativas para avaliar, em baixa latência, a taxa de processamento interna β_i dos pontos voluntários sem módulos adicionais comerciais. Ele assume, na configuração padrão, que os pontos são homogêneos e confiáveis, o que exige cautela quando a disponibilidade é intermitente. [14]

3.2.4.2 Traefik

O Traefik representa uma geração mais moderna de balanceadores, desenhada para microsserviços e orquestração “dinâmica”. Tal orquestração é diferente de dizer que o método é dinâmico, como descrito em 3.2.2. A importância do Traefik no mercado é graças à sua simplicidade, não apenas técnica, quanto também de configuração. Em Nginx existe apenas um método de balanceamento dinâmico, enquanto em Traefik não existem implementações anteriores nem abertura nas configurações para tal. [15]

3.2.4.3 HAProxy

O HAProxy (*High Availability Proxy*) é a referência de mercado para balanceamento de carga TCP/HTTP de alto desempenho. Sua relevância para o problema proposto reside na alta configurabilidade dos algoritmos de balanceamento (e uma série de algoritmos já implementados) [16]. Caso se deseje implantar um método dinâmico, HAProxy é a decisão entre as soluções apresentadas. Adicionalmente, quando comparado com Nginx, é em média superior em escalabilidade, latência e taxa de erros [17].

Sua performance, baixo custo computacional e configurabilidade tornam o HAProxy uma escolha robusta para cenários similares aos do problema deste trabalho, onde os recursos são heterogêneos e é impossível ter informações confiáveis

sobre os pontos.

3.3 Blockchain

A tecnologia *Blockchain* transcendeu a sua aplicação original em criptomoedas para se tornar um pilar fundamental em arquiteturas distribuídas modernas. Sua relevância para este trabalho reside na capacidade de estabelecer consenso e garantir a integridade de dados em ambientes onde os participantes não possuem confiança mútua prévia (*zero-trust*). Ao analisarmos a evolução desta tecnologia, buscamos compreender os mecanismos criptográficos que permitem a coordenação segura de nós dispersos, separando-os dos modelos financeiros que frequentemente os acompanham.

Historicamente, a primeira barreira para a descentralização confiável foi a prevenção de abusos e ataques de negação de serviço (DoS) e ataques Sybil. O primeiro vislumbre para resolver o problema foi proposto por Adam Back com o HashCash, introduzindo a ideia de Prova de Trabalho (*Proof-of-Work* - PoW), com o intuito de tornar inviável o custo computacional para fraudar dados distribuídos ou sobrecarregar uma rede [18].

Contudo, faltava um mecanismo de incentivo persistente para manter os participantes honestos à longo prazo. Essa questão foi resolvida por Satoshi Nakamoto com a criação do Bitcoin. Ao combinar a estrutura de PoW com blocos imutáveis, criou-se a *Blockchain*, introduzindo o conceito de *tokens* que incentivam financeiramente a permanência honesta e a colaboração na rede.

A partir do sucesso do Bitcoin, a academia e a indústria buscaram expandir esse modelo para fora das transações financeiras, visando a computação generalista. Zyskind et al. demonstraram que, utilizando técnicas de armazenamento *off-chain* combinadas com uma blockchain para controle de acesso, seria possível executar códigos complexos e armazenar dados em nós não confiáveis garantindo a privacidade e integridade [19].

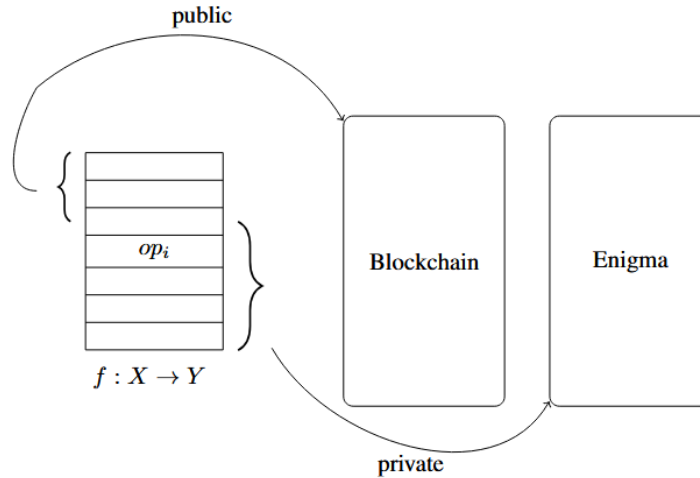


Figura 3: Enigma Code Execution model [19]

Essa linhagem evolutiva culminou no atual cenário de Redes de Infraestrutura Física Descentralizada (DePIN), onde o consenso é mantido por complexos modelos econômicos (*tokenomics*). Embora eficazes para criar mercados abertos, esses modelos impõem barreiras financeiras e volatilidade especulativa, o que diverge do objetivo deste trabalho.

Portanto, enquanto os trabalhos relacionados utilizam a blockchain para gerir pagamentos e consenso global, a nossa proposta busca isolar as primitivas de segurança sem atrelá-las a um sistema monetário. O objetivo é recuperar o espírito de colaboração voluntária, aplicando a segurança moderna nos dados compartilhados para mitigar a desconfiança em ambientes distribuídos.

3.4 Trabalhos acadêmicos similares

Embora a computação voluntária para fins científicos esteja madura 3.1, a utilização de recursos voluntários para hospedagem de serviços web com garantia de qualidade de serviço (QoS) é um campo de pesquisa ativo, normalmente referenciado na literatura como *Volunteer Cloud*, com um dos projetos precursores sendo Cloud@Home[20].

Anderson e Fedak (2006) [21] estabeleceram a base fundamental para esta área ao analisar medições de mais de 330.000 *hosts* em projetos de computação voluntária. O estudo demonstrou que o potencial desses recursos excede largamente as aplicações de simples processamento de CPU (como no SETI@home), possuindo capacidade suficiente para suportar aplicações intensivas em dados e armazenamento. Esta validação quantitativa sustenta a premissa deste trabalho de que a infraestrutura ociosa existente é suficiente para hospedar serviços web completos, como é possível identificar pelo crescimento da curva cinza da Figura 4, indicando o aumento da ociosidade nos hosts.

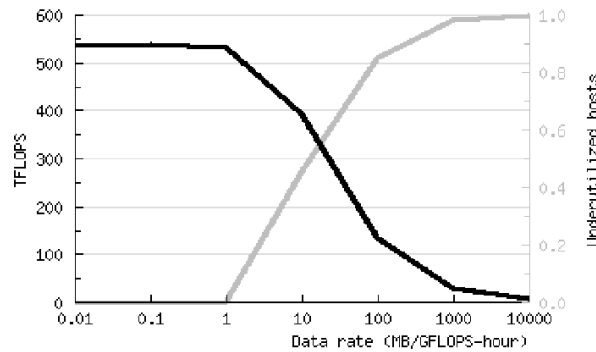


Figura 4: Computing power versus data rate [21]

Culoso et al. (2009) formalizaram o conceito de Cloud@Home, propondo uma arquitetura onde a infraestrutura de nuvem é formada inteiramente por nós voláteis e não confiáveis [20][22]. A principal contribuição desses trabalhos foi identificar que, para garantir a disponibilidade de serviços *web* em um ambiente no qual nós se desligam aleatoriamente (*churn*), é necessária uma gestão agressiva de redundância e previsão de disponibilidade, diferentemente do modelo de tarefas assíncronas da computação voluntária como no BOINC.

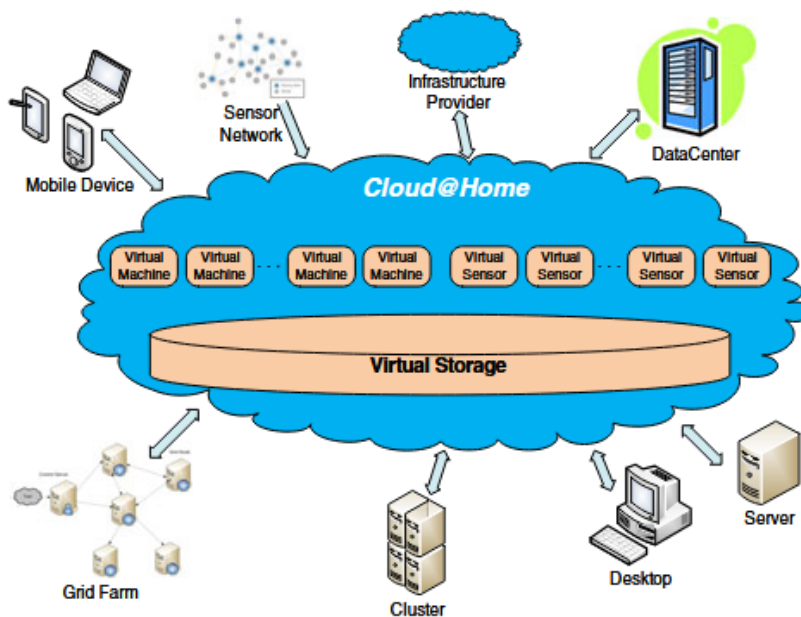


Figura 5: Cenário de referência Cloud@home [22]

Paralelamente, Chard et al. (2010) propuseram uma arquitetura de compartilhamento de recursos baseada em relacionamentos sociais, em contraposição aos modelos focados em lógicas de mercado [23]. A premissa da Social Cloud é utilizar a confiança pré-existente entre instituições ou redes sociais (como Facebook à época) para mitigar a necessidade de incentivos financeiros explícitos.

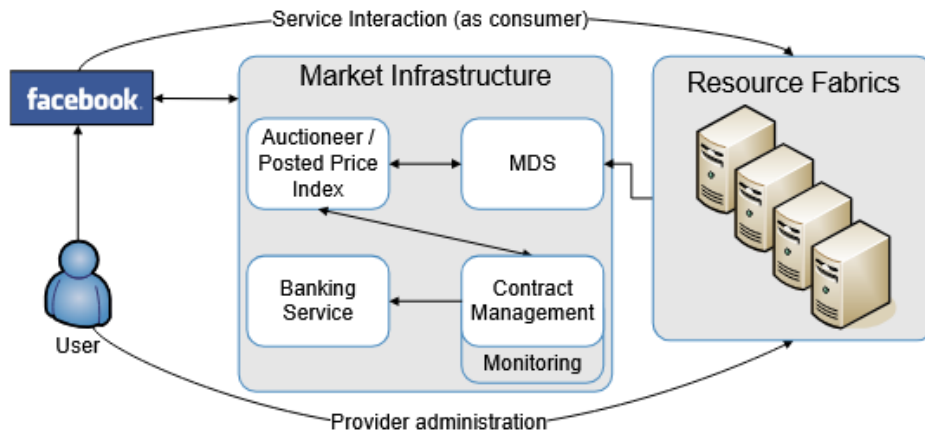


Figura 6: Sistema de Autenticação proposto na Social Cloud [23]

Avançando para implementações modernas, Tang, Fedak et al. (2022) propuseram o WukaStore, um framework de armazenamento híbrido que integra nuvens estáveis de grandes provedores (como AWS) com armazenamento volátil de desktops voluntários. O estudo comprovou via simulações e deploys na plataforma Grid'5000, que a combinação de recursos voluntários instáveis com uma camada de orquestração pode oferecer serviços de alta disponibilidade e durabilidade, reduzindo enormemente os custos de armazenamento de *Big Data* [24].

Embora o WukaStore valide o modelo híbrido (nuvem + voluntário) para armazenamento de *Big Data*, o nosso trabalho se diferencia ao focar na hospedagem de serviços *web* síncronos, com tolerância à latência menor e tratamento em tempo real das requisições http.

3.5 Soluções de Mercado e DePINs

No cenário comercial, a aplicação prática desses conceitos acadêmicos convergiu em produtos com diversas abordagens para oferecer serviços de nuvem descentralizada. Nesta subseção abordaremos as Redes de Infraestrutura Física Descentralizada (DePIN), que utilizam *blockchain* para incentivar a oferta de recursos, assim como outras propostas que não são baseadas em sistema de *tokens*.

3.5.1 Computação e Hospedagem Descentralizadas

A Akash Network, que evoluiu de um artigo proposto em 2018 [25] e faz uma orquestração de *containers* descentralizada, é uma DePIN baseada em Kubernetes [26], com 60 provedores ativos em novembro de 2025. Operando como um mercado de computação em nuvem, a Akash utiliza um modelo no qual locatários especificam requisitos e provedores competem para hospedar os serviços encapsulados em Docker *containers*[27]. A plataforma resolve o problema de conectividade por meio de uma rede segura, permitindo que provedores domésticos exponham serviços *web* publicamente [25]. A lógica comercial é baseada no *token* nativo (\$AKT), usando uma *blockchain* para registro das transações.

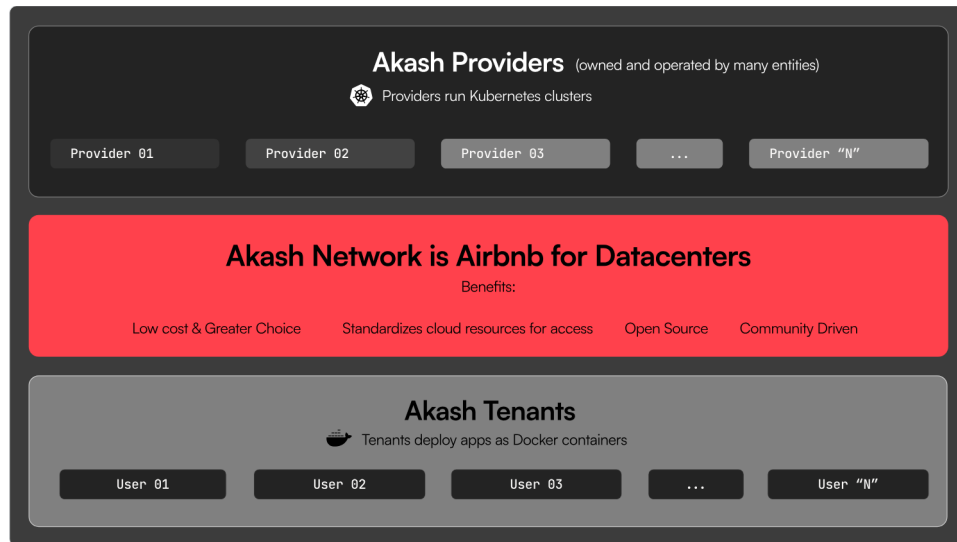


Figura 7: Visão Geral da Arquitetura Akash [28]

A rede Flux se apresenta como uma alternativa aos grandes provedores, propondo uma nuvem descentralizada baseada em tokenização. Um provedor pode criar um *Flux Node* para receber recompensas em tokens, incentivando a manutenção da rede, com aproximadamente 7600 nós em novembro de 2025, enquanto usuários da *Flux Cloud* pagam pelo uso da rede também em *tokens* proprietários[29][30].

Já o Golem Network, proposto por Janiuk et al. em 2016, opera a venda de poder de processamento *p2p*. Originalmente focado em tarefas de renderização em lote (*batch processing*), o Golem evoluiu para uma arquitetura generalista ("Yagna") que permite a execução de tarefas arbitrárias em ambientes isolados [31]. Sua arquitetura de segurança baseia-se em um sistema de reputação e verificação redundante, onde o resultado de uma computação pode ser validado por múltiplos provedores antes do pagamento ser liberado. Um modelo que serve para entendermos a lógica de reputação em sistemas descentralizados.

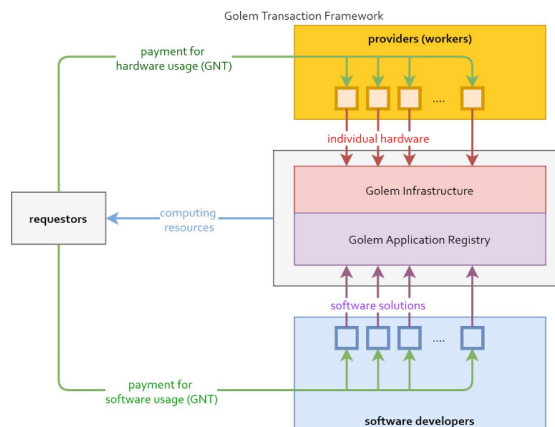


Figura 8: Arquitetura Golem Network [31]

3.5.2 Armazenamento e Distribuição

O armazenamento de dados em redes *zero-trust* exige estratégias robustas de criptografia e fragmentação para garantir privacidade e durabilidade. A seguir discutiremos como soluções de mercado abordam este tópico.

O InterPlanetary File System (IPFS) destaca-se como a camada de distribuição da *web* descentralizada, utilizando endereçamento de conteúdo (*content-addressing* - CID), onde arquivos são identificados por seus hashes criptográficos[32].

Trautwein et al. (2022) demonstram que o IPFS opera sobre uma tabela de Hash Distribuída (DHT) baseada no protocolo Kademlia [33]. Proposto originalmente por Maymounkov e Mazières (2002), o Kademlia utiliza a métrica XOR para otimizar a busca de nós ($O(\log n)$), permitindo que o IPFS localize conteúdo em uma rede global sem servidores centrais [34].

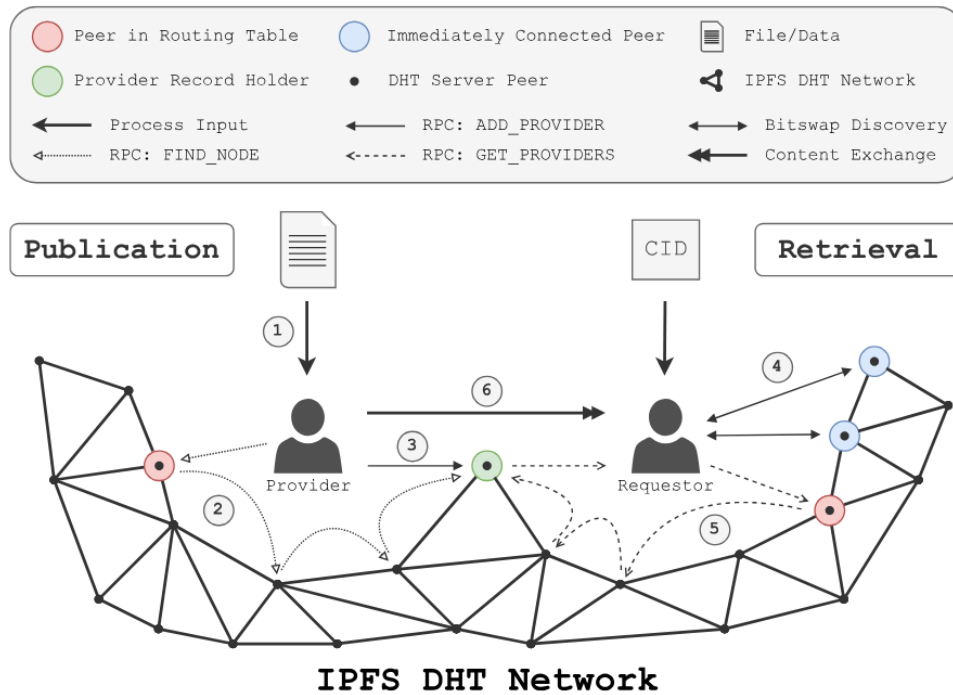


Figura 9: Modelo de Publicação e busca de dados usando IPFS [33]

Sia oferece uma arquitetura de armazenamento em nuvem baseada em contratos inteligentes. Vorick e Champine (2014) descrevem um protocolo onde locatários formam contratos de armazenamento com hospedeiros, garantidos por provas de armazenamento (*Proof-of-Storage*) enviadas periodicamente à *blockchain* [35]. Os dados são criptografados e distribuídos, garantindo que nenhum hospedeiro tenha acesso ao conteúdo original. Em novembro de 2025, conta com 548 hosts ativos e 14200 contratos usando a moeda proprietária baseada em *tokens* Siacoin[36].

Similarmente, a Storj propõe um modelo que prioriza a performance no modelo p2p. Wilkinson et al. (2014) detalham o uso de *Erasure Coding* (código de apagamento), onde um arquivo é fragmentado em n partes, das quais apenas k são necessárias para a reconstrução ($k;n$) [37]. Isso permite que a rede tolere o desligamento simultâneo de múltiplos nós voluntários (*churn*) sem perda de dados, otimizando a latência ao baixar fragmentos de múltiplos pares simultaneamente, uma técnica superior à replicação simples utilizada em sistemas tradicionais.

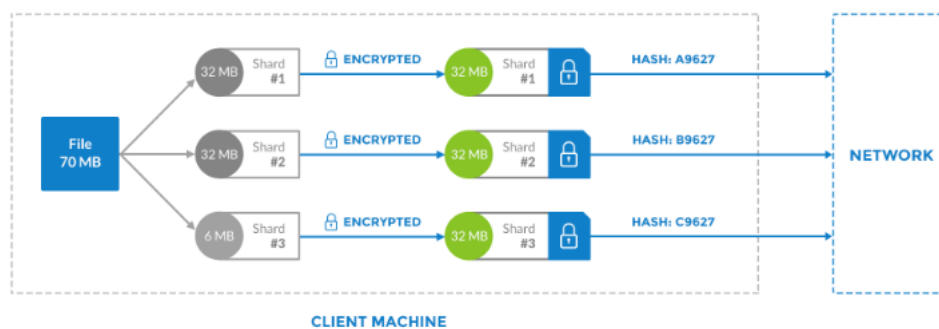


Figura 10: Processo de fragmentação (*Erasure Coding*) [37]

4 Objetivos específicos

4.1 Entregas

4.1.1 Serviço web para testes

Dentre as entregas, esta é provavelmente a mais simples. Deve-se construir um serviço web de exemplo, o qual aplicaremos nossa arquitetura modular sobre. Apesar da dita simplicidade técnica, este serviço deve se encontrar em um equilíbrio construído com cuidado: simples o bastante para não adicionar complexidades não condizentes com aplicações normais, mas completo o bastante para poder testar aprimoramentos que só fazem diferença em face de certa complexidade encontrada em aplicações reais.

Um exemplo de tal caso é o uso de *cache* juntamente com balanceamento de carga. Se nosso serviço web de teste contém apenas uma página estática, ao aplicar *caching* não veremos grandes diferenças entre os testes com e sem balanceamento de carga e, dependendo da configuração, podemos ver uma depreciação com a adição do balanceamento. Este efeito ocorreria pois o *cache* sozinho diminui substancialmente a latência, pois a busca em memória é mais rápida que o redirecionamento TCP e, logicamente, mais rápido que a busca em disco.

Assim, deste simples exemplo, consegue-se definir que o serviço web deve conter uma alta quantidade de páginas com conteúdo para que possa-se visualizar certos efeitos do balanceamento de cargas que buscamos. É além dos objetivos deste trabalho e também contraprodutivo tentar definir os requisitos mínimos do serviço web para testes; os requisitos surgirão naturalmente durante o desenvolvimento e configuração dos sistemas. Nestes termos, basta definir que o desenvolvimento do serviço deve se ater às complexidades encontradas comumente na indústria.

4.1.2 Balanceador de carga configurado

Como discutido na seção 3, o balanceador de carga de código aberto HAProxy se encaixa às necessidades da solução buscada. Sua configuração (subsequentemente, também a exposição de alto nível de uma porção das configurações para os usuários finais) e os experimentos para escolhas de parâmetros conferem ao trabalho uma complexidade digna, mesmo ao usar códigos prontos.

O balanceador de carga deve utilizar eficientemente métodos dinâmicos de balanceamento (3.2.2), buscando o mínimo de latência adicionada pelo *overhead*.

4.1.3 Sistema de inscrição de voluntários

Outro subsistema importante da solução é o sistema de inscrição dos servidores voluntários, onde eles podem entrar na lista de servidores disponíveis e obter a cópia do serviço web. Duas soluções foram preliminarmente estudadas: a primeira, com as informações do servidor principal e o serviço web encapsulados em um *container* e a segunda, com uma rota especializada no servidor principal que conecta o servidor voluntário e envia a cópia do serviço. A primeira solução sofre com uma perda de performance por virtualização no cliente e a segunda sofre com uma perda de performance no servidor. Soluções de meio-caminho como uma cópia do serviço web com execução nativa e um código para a inscrição sofrem com prováveis falhas humanas. Para tal, a primeira solução vai ser utilizada deste ponto adiante.

4.1.3.1 Lado do servidor

Ao lado do servidor, a parte básica desse subsistema é a adição do novo servidor voluntário ao balanceador de carga. Verificações de qualidade de conexão, de prontidão da configuração do cliente e da confiabilidade de dados são adiões de

qualidade ao sistema. É de suma importância que o subsistema seja leve, então uma aplicação compilada para execução nativa, escrita em *C* ou *Rust*, é recomendável.

4.1.3.2 Lado do cliente

Quanto ao lado do cliente, é esperado que estejam encapsulados em um *container* o serviço web descrito anteriormente e o módulo de inscrição que corresponde ao sistema do servidor descrito anteriormente. O subsistema deve corretamente identificar o servidor em que está inserido e enviar as informações necessárias para o servidor.

4.1.4 Túnel principal-voluntário e sua arquitetura

Como pode-se esperar, é necessário estabelecer uma conexão de rede entre o servidor principal e os voluntários, preferencialmente sem alterações na configuração de rede do cliente. Desta forma, deve-se planejar e implementar um túnel. O túnel pode ser implementado através de WebSockets, gRPC, QUIC ou até conexões UDP/TCP puras.

Assim como a implementação do sistema de inscrição de voluntários no lado do servidor, o *overhead* deve ser minimizado, afinal esta é uma parte essencial do , presente em toda e qualquer requisição. Novamente, implementações eficientes e leves em *C* ou *Rust* são bem-vindas.

4.1.5 Extra: Sistema de validação e sua arquitetura

Um subsistema essencial para a confiança de dados é o sistema de validação que confere os retornos das requisições dos voluntários. Uma solução trivial proposta é uma assinatura digital que contém uma *hash* dos dados criptografada a chave privada do servidor principal, que pode então ser descriptografada e validada pelos usuários com a chave pública.

4.1.6 Extra: Banco de dados com proteção contra leitura e sua arquitetura

Caso o projeto evolua o bastante, para maior abrangência de casos de uso é importante implementar um banco de dados (ou um sistema de operações) capaz de proteger dados sensíveis que não podem ser expostos aos voluntários. Uma proposta foi diagramada e exposta na Figura 11, onde operações seriam realizadas sobre linhas criptografadas e os segredos compartilhados apenas entre o principal e o usuário.

4.2 Experimentos e Pesquisas

Para realizar escolhas ideais de parâmetros, busca de falhas e verificar os resultados esperados, faz-se necessário praticar rodadas de experimentos e pesquisas.

4.2.1 Pesquisa de interesse e confiabilidade

É de interesse da relevância do trabalho validar o que foi assumido na Seção 2 sobre a abertura de indivíduos e instituições públicas quanto o empréstimo de computação ociosa. De acordo com os resultados e as interpretações, é possível mover o foco e estudar maneiras de mitigar barreiras humanas contra a implantação da arquitetura.

4.2.2 Comparação de diferentes algoritmos de balanceamento

Devemos testar sobre múltiplas formas de estresse as configurações do balanceador para buscar a configuração ótima. Variáveis a ser controladas, tanto em voluntários como no principal, são:

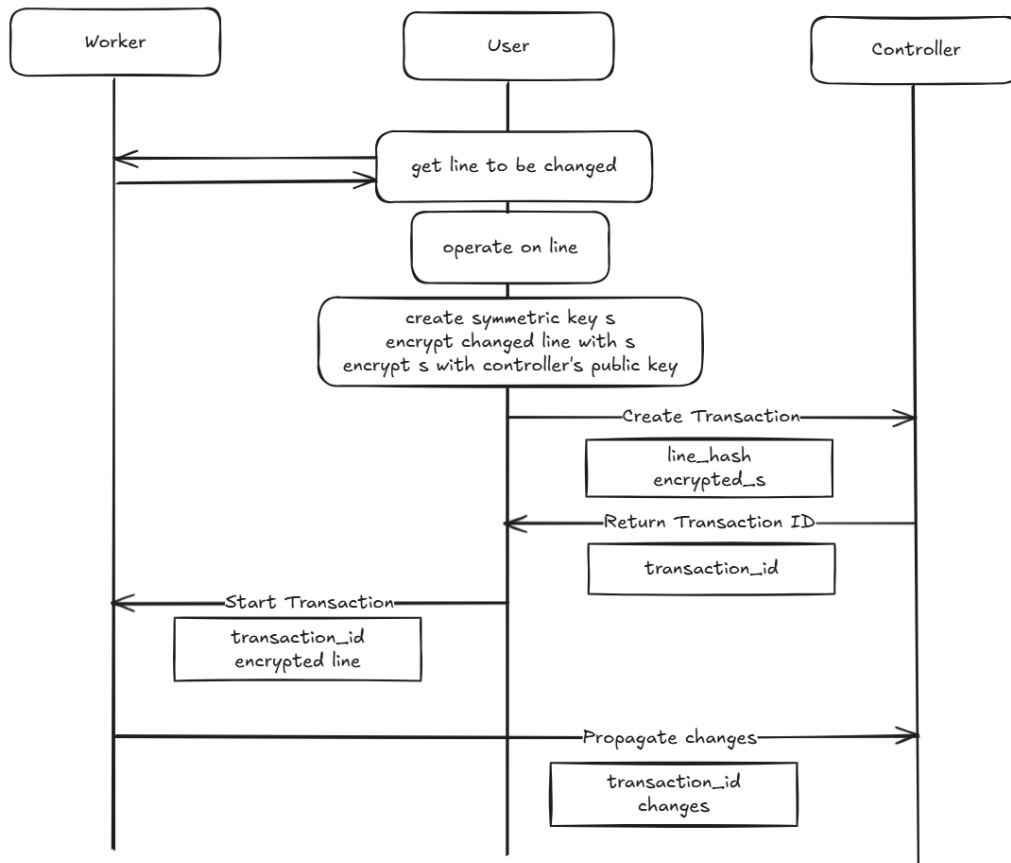


Figura 11: Proposta para Banco de Dados: proteção através de criptografia de linha. Fonte: autores

- Consumo mínimo, médio e máximo de CPU;
- Consumo mínimo, médio e máximo de memória;
- Latência mínima, média e máxima;
- Taxa mínima, média e máxima de dados;
- Taxa de erros por requisição;
- Taxa de requisições completas.

4.2.3 Comparação de teste de carga com soluções alternativas

Finalmente, deve-se estudar como a arquitetura se compara ao estado da arte no mercado. Deve-se, então, encontrar configurações comparáveis em provedores de computação em nuvem e, caso aplicável, configurações tradicionais de balanceamento de carga.

Dados estatísticos sobre o desempenho e gráficos de evolução de carga contra latência e erros são o produto final desta comparação.

5 Materiais e métodos

Esta seção detalha os recursos computacionais, as ferramentas de *software* e os procedimentos experimentais adotados para o desenvolvimento da arquitetura proposta e para a validação das hipóteses levantadas. A metodologia divide-se em três etapas principais: a construção do ambiente de testes, a pesquisa de aceitação (fator humano) e os experimentos de desempenho (fator técnico).

5.1 Materiais e ferramentas

Para garantir a fidelidade aos requisitos de eficiência, foram selecionados os seguintes recursos:

5.1.1 Hardware e ambiente de virtualização

Os experimentos serão conduzidos em um ambiente controlado, projetado para simular a heterogeneidade de rede e *hardware* esperada entre o servidor principal e os voluntários.

- Servidor principal: máquina local com recursos limitados (≤ 4 GB de RAM e processador ≤ 2 cores), rodando sistema operacional Linux (Ubuntu *server* é uma opção);
- Pontos de computação voluntários: conjunto de máquinas (virtuais ou físicas) com configurações de hardware variadas e posições geográficas distantes para testar em situação de heterogeneidade;
- Gerador de carga: máquina dedicada, externa ao cluster, para execução dos *scripts* de teste de estresse, assegurando que a instrumentação da medição não impacte o desempenho do sistema avaliado.

5.1.2 Software e tecnologias

A *stack* foi escolhida visando alto desempenho, baixo *overhead* e otimização de trabalho:

- Balanceamento de carga: HAProxy (a versão LTS mais recente na época de construção), escolhido por sua robustez e suporte a algoritmos dinâmicos configuráveis;
- Linguagens de desenvolvimento:
 - *Rust* ou *C*: para o desenvolvimento do sistema de inscrição e do túnel de comunicação, devido à necessidade de *overhead* mínimo;
 - *JS*: para o desenvolvimento do serviço web de teste, permitindo rápida prototipagem de endpoints.
- Containerização: Docker, utilizado para encapsular o serviço web e o módulo de cliente nos nós voluntários;
- Ferramentas de teste de carga: JMeter, selecionado por sua capacidade de gerar relatórios detalhados de latência e suporte a conexões simultâneas massivas e experiência prévia dos autores;
- Monitoramento: Prometheus e ferramentas nativas para coleta de métricas de CPU, RAM e rede em tempo real.

5.2 Procedimentos metodológicos

5.2.1 Pesquisa de interesse e confiabilidade

Para validar a viabilidade social do projeto (empréstimo de computação ociosa), será aplicado um questionário estruturado quantitativo.

- Público-alvo: profissionais de TI, estudantes universitários e gestores de infraestrutura de instituições públicas.
- Instrumento: formulário online contendo perguntas em escala Likert (1-5) sobre a disposição em instalar o software de empréstimo de computação ociosa e preocupações com segurança/privacidade.
- Análise: os dados serão tabulados para identificar a correlação entre o nível de conhecimento técnico do usuário e sua predisposição a aceitar a instalação do *container* de empréstimo de computação ociosa.

5.2.2 Experimentos de balanceamento e comparação

5.2.2.1 Balanceamento

Os experimentos de balanceamento de carga dependem largamente de quais configurações é possível ter opções diversas. Por exemplo, não faz sentido testar estaticamente sabendo que os algoritmos estáticos não têm aplicação no projeto. Igualmente, exceto por motivos de avaliação de *overhead* de módulos, faz pouco sentido testar configurações sem segurança pois não são aplicáveis.

5.2.2.2 Comparação com outras soluções

Para determinar comparar com soluções de mercado, são previstos testes de estresse em três cenários distintos:

- Aplicação rodando em nuvem com *autoscaler*;
- Arquitetura clássica, local com vários servidores;
- Arquitetura orientada à voluntários construída.

5.2.2.3 Protocolo

Todos testes submeterão rotas (distintas caso faça sentido haver mais de uma, mas iguais entre os cenários) a uma carga incremental de requisições por um período determinado. Durante a execução, serão coletadas as seguintes métricas a cada segundo:

1. Consumo de recursos (CPU/Memória) no servidor principal;
2. Latência da requisição;
3. Taxa de sucesso.

6 Resultados esperados

Os resultados apresentados nesta seção consistem em valores de referência e estruturas de relatório previstas para análises futuras. Como os experimentos ainda não foram realizados, todos os dados aqui descritos representam metas de qualidade ou faixas de operação esperadas com base em literatura, práticas comuns ou requisitos do projeto.

6.1 Pesquisa de interesse e confiabilidade

A Tabela 1 apresenta as medidas esperadas para a aplicação do questionário de aceitação. Os valores representam referências de qualidade para análise posterior. De forma geral, estes valores representariam uma confirmação do que foi assumido na Seção 2.

Tabela 1: Valores de referência desejados na pesquisa de aceitação

Métrica	Valor de referência
Média de intenção de uso (Likert 1–5)	≥ 3.5
Pontuação média entre usuários técnicos	≥ 4.0

6.2 Experimentos de balanceamento de carga

A Tabela 2 lista indicadores desejáveis para os testes de balanceamento utilizando HAProxy e pontos de computação voluntários.

Tabela 2: Valores de referência para testes de balanceamento

Métrica	Valor de referência
Desvio padrão da distribuição de carga	$\leq 10\text{--}15\%$
Overhead adicional por requisição	$\leq 5\text{--}10\text{ ms}$
Latência média	$\leq 500\text{ ms}$

6.3 Comparação entre arquiteturas

Os valores de referência para os três cenários planejados (nuvem com autoscaler, arquitetura tradicional e arquitetura orientada à voluntários) estão descritos na Tabela 3.

Tabela 3: Valores de referência para comparação entre cenários

Métrica	Nuvem	Tradicional	Voluntários
Latência média (ms)	50–150	200–250	$\leq 200\text{--}250$
Taxa de sucesso	100%	$\geq 98\%$	$\geq 98\%$

6.4 Métricas gerais previstas

Para uniformidade entre todos os experimentos, a coleta será baseada nas métricas mostradas na Tabela 4.

Tabela 4: Faixas de referência para métricas gerais

Métrica	Faixa de referência
Memória do servidor principal	$\leq 3\text{ GB}$
Latência P95	$\leq 200\text{ ms}$
Taxa mínima de sucesso	$\geq 98\%$

7 Cronograma de atividades do TCC02

F.A: Configuração Mínima
Primeira fase dedicada à construção da versão inicial do sistema, com serviços básicos funcionando e validação da comunicação entre os voluntários e o servidor principal.
Atividades planejadas para a fase A
A.1 Construção do serviço web A.2 Configuração funcional do balanceador de carga A.3 Sistema de inscrição de voluntários (versão mínima) A.4 Arquitetura túnel principal–voluntário A.5 Implementação do túnel principal–voluntário
Resultados esperados para a fase A
Versão inicial e funcional do sistema, com comunicação básica entre voluntários e servidor, serviço web operacional e infraestrutura mínima configurada. Esses resultados demarcam o fim da fase.
F.B: Coarse Tuning
Fase voltada à evolução da versão inicial do sistema, aprofundando complexidade, avaliando desempenho e identificando pontos de otimização.
Atividades planejadas para a fase B
B.1 Iteração do serviço web com aumento de complexidade B.2 Benchmark de configurações do balanceador de carga B.3 Benchmark do sistema de inscrição de voluntários B.4 Relatório de melhorias
Resultados esperados para a fase B
Relatório consolidado com os resultados dos benchmarks e lista extensa de melhorias. Esses resultados demarcam o fim da fase.
F.C: Fine Tuning
Fase de refinamento fino, implementação aprofundada de melhorias e introdução do sistema de validação.
Atividades planejadas para a fase C
C.1 Implementação de melhorias identificadas no relatório C.2 Iterações suplementares de benchmark C.3 Análise e atualização do relatório com resultados adicionais C.4 Arquitetura do sistema de validação C.5 Implementação do sistema de validação
Resultados esperados para a fase C
Sistema otimizado com validação integrada e melhorias finas aplicadas, garantindo maior qualidade e desempenho. Esses resultados demarcam o fim da fase.
F.D: Finalização
Última fase, dedicada à implementação dos componentes finais e testes globais do sistema.
Atividades planejadas para a fase D
D.1 Arquitetura do banco de dados ofuscado D.2 Implementação do banco de dados ofuscado

Resultados esperados para a fase D

Sistema concluído com banco de dados ofuscado implementado, testes finais executados e preparação para entrega e apresentação do TCC02. Esses resultados demarcam o fim da fase.

2026					
01	02	03	04	05	06
F.A: A.1					
F.A: A.2					
F.A: A.3					
	F.A: A.4				
	F.A: A.5				
		F.B: B.1			
		F.B: B.2			
		F.B: B.3			
		F.B: B.4			
			F.C: C.1		
			F.C: C.2		
			F.C: C.3		
			F.C: C.4		
				F.C: C.5	
				F.D: D.1	
				F.D: D.2	
					F.D: D.3

Referências Bibliográficas

- [1] Canalys, “Cloud Infrastructure Spend Grows 46% in Q4 2018 to Exceed US\$80 Billion for Full Year,” Press Release, February 2019, acessado em: 29 nov. 2025. [Online]. Available: https://www.canalys.com/static/press_release/2019/pr20190204.pdf
- [2] Cloud Communications, “Worldwide cloud service spending to grow by 19% in 2025,” February 2025, acessado em: 29 nov. 2025. [Online]. Available: <https://www.cloudcommunications.com/news/worldwide-cloud-service-spending-to-grow-by-19-in-2025>
- [3] OpenStreetMap Foundation, “Finances,” <https://osmfoundation.org/wiki/Finances>, 2025, acessado em: 29/11/2025.
- [4] ProPublica, “Internet Archive Tax Filings,” <https://projects.propublica.org/nonprofits/organizations/943242767>, acessado em: 29/11/2025.
- [5] Wikimedia Foundation, “Financial Reports,” <https://wikimediafoundation.org/who-we-are/financial-reports>, 2025, acessado em: 29/11/2025.
- [6] Mozilla Foundation and Subsidiaries, “Independent Auditors’ Report and Consolidated Financial Statements, December 31, 2023 and 2022,” Audit Report by Hood & Strong, 2024, acessado em: 29/11/2025. [Online]. Available: <https://assets.mozilla.net/annualreport/2024/mozilla-fdn-2023-fs-final-short-1209.pdf>
- [7] BBB Wise Giving Alliance, “Give.org Donor Trust Report 2023: Donor Participation Special Report,” Give.org, 2023, acessado em: 29/11/2025. [Online]. Available: <https://give.org/donor-trust-report>
- [8] S&P Global Market Intelligence, “2025 trends in cloud and cloud native,” S&P Global, Tech. Rep., December 2024, licensed to AMD. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/solutions/data-center/insights/2025-trends-cloud-native-vanguard-report.pdf>
- [9] Great Internet Mersenne Prime Search (GIMPS), “The Great Internet Mersenne Prime Search (GIMPS) Home Page,” <https://www.mersenne.org/>, 2025, acessado em: 29/11/2025.
- [10] BOINC, “BOINC Wiki,” <https://boinc.berkeley.edu/wiki>, 2025, acessado em: 29/11/2025.
- [11] G. Fedak, C. Germain, V. Neri, and F. Cappello, “Xtremweb: a generic global computing system,” in *Proceedings First IEEE/ACM International Symposium on Cluster Computing and the Grid*, 2001, pp. 582–587.
- [12] H. Kameda, J. Li, C. Kim, and Y. Zhang, *Optimal load balancing in distributed computer systems*, 1st ed., ser. Telecommunication Networks and Computer Systems. Guildford, England: Springer, Dec. 1996.
- [13] T. Kaistha and K. Ahuja, “Imperative load-balancing techniques in heterogeneous wireless networks,” in *Comprehensive Guide to Heterogeneous Networks*. Elsevier, 2023, pp. 249–269.
- [14] NGINX, Inc., “HTTP Load Balancing,” https://nginx.org/en/docs/http/load_balancing.html, 2025, acessado em: 29/11/2025.
- [15] Traefik Labs, “Traefik Documentation,” <https://doc.traefik.io/>, 2025, acessado em: 29/11/2025.
- [16] HAProxy Technologies, “HAProxy Documentation,” <https://docs.haproxy.org/>, 2025, acessado em: 29/11/2025.
- [17] L. H. Pramono, R. C. Buwono, and Y. G. Waskito, “Round-robin algorithm in HAProxy and nginx load balancing performance evaluation: A review,” in *2018 International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*. IEEE, Nov. 2018, pp. 367–372.
- [18] A. Back, “Hashcash - A Denial of Service Counter-Measure,” <http://www.hashcash.org/papers/hashcash.pdf>, 2002.
- [19] G. Zyskind, O. Nathan, and A. Pentland, “Enigma: Decentralized Computation Platform with Guaranteed Privacy,” 2015. [Online]. Available: <https://arxiv.org/abs/1506.03471>

- [20] V. Cunsolo, S. Distefano, A. Puliafito, and M. Scarpa, “Volunteer computing and desktop cloud: The cloud@home paradigm,” in *Volunteer Computing and Desktop Cloud: The Cloud@Home Paradigm*, 08 2009, pp. 134 – 139.
- [21] D. P. Anderson and G. Fedak, “The computational and storage potential of volunteer computing,” in *Proceedings of the 6th IEEE International Symposium on Cluster Computing and the Grid (CCGRID’06)*, vol. 1. IEEE, 2006, pp. 73–80.
- [22] S. Distefano, V. D. Cunsolo, A. Puliafito, and M. Scarpa, *Cloud@Home: A New Enhanced Computing Paradigm*. Boston, MA: Springer US, 2010, pp. 575–594. [Online]. Available: https://doi.org/10.1007/978-1-4419-6524-0_25
- [23] K. Chard, S. Caton, O. Rana, and K. Bubendorfer, “Social cloud: Cloud computing in social networks,” in *2010 IEEE 3rd International Conference on Cloud Computing*. IEEE, 2010, pp. 99–106. [Online]. Available: https://www.researchgate.net/publication/221399959_Social_Cloud_Cloud_Computing_in_Social_Networks
- [24] B. Tang and G. Fedak, “Wukastore: Scalable, configurable and reliable data storage on hybrid volunteered cloud and desktop systems,” *IEEE Transactions on Big Data*, vol. PP, pp. 1–1, 10 2017. [Online]. Available: <https://doi.org/10.1109/TBDATA.2017.2758791>
- [25] Akash Network, “The akash network whitepaper,” <https://akash.network/whitepaper>, 2020, acessado: 30/11/2025.
- [26] The Kubernetes Authors, “Concepts overview,” <https://kubernetes.io/docs/concepts/overview/>, Kubernetes, accessed: 30 November 2025.
- [27] Docker Inc., “Docker documentation,” <https://docs.docker.com/>, Docker, accessed: 30 November 2025.
- [28] Akash Network, “General information,” <https://akash.network/about/general-information/>, Akash Network, accessed: 30 November 2025.
- [29] The Flux Authors, “Flux ecosystem,” <https://docs.runonflux.com/home/flux-ecosystem>, Flux, accessed: 30 November 2025.
- [30] The Flux Team, “Flux Whitepaper v1.1: An in-depth description of the Flux Ecosystem and its novel subcomponents,” <https://www.cryptocompare.com/media/39456580/fluxwhitepaper.pdf>, 2021, accessed: 30 November 2025.
- [31] P. Janiuk, A. Skrzypczak, A. Regulski, and J. Zawistowski, “The golem project: The golem network whitepaper,” <https://golem.network/doc/Golemwhitepaper.pdf>, 2016, acessado: 30/11/2025.
- [32] J. Benet, “IPFS - Content Addressed, Versioned, P2P File System (DRAFT 3),” <https://ipfs.io/ipfs/QmR7GSQM93Cx5eAg6a6yRzNde1FQv7uL6X1o4k7zrJa3LX/ipfs.draft3.pdf>, 2014, version: DRAFT 3; Original publication on arXiv: <http://arxiv.org/abs/1407.3561>.
- [33] D. Trautwein, A. Raman, G. Tyson, I. Castro, W. Scott, M. Schubotz, B. Gipp, and Y. Psaras, “Design and Evaluation of IPFS: A Storage Layer for the Decentralized Web,” in *Proceedings of the ACM SIGCOMM 2022 Conference (SIGCOMM ’22)*. Amsterdam, Netherlands: ACM, 2022, pp. 739–752. [Online]. Available: doi.org/10.1145/3544216.3544232
- [34] P. Maymounkov and D. Mazières, “Kademlia: A Peer-to-peer Information System Based on the XOR Metric,” in *International Workshop on Peer-to-Peer Systems (IPTPS)*, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 99–108. [Online]. Available: <https://www.scs.stanford.edu/~dm/home/papers/kpos.pdf>
- [35] D. Vorick and L. Champine, “Sia: Simple decentralized storage,” <https://sia.tech/whitepaper.pdf>, 2014, nebulous Inc.
- [36] SiaGraph Contributors, “SiaGraph: Sia Network Statistics,” <https://siagraph.info/>, Sia Network, accessed: 30 November 2025.
- [37] S. Wilkinson, T. Boshevski, J. Brandoff, and V. Buterin, “Storj: A peer-to-peer cloud storage network,” <https://static.storj.io/storj2014.pdf>, dec 2014, version 1.01.